

TPFI 2025/26

Hw 1: Programmazione su Liste & Ordine Superiore

docente: I. SALVO – Sapienza Università di Roma

assegnato: 18 marzo 2026, consegna 31 marzo 2026

Nota: Consegnare un unico file testo con estensione `.hs` (preferibilmente di nome `HW1-NomeCognome.hs`). Saranno apprezzate soluzioni ‘composizionali’ nello stile funzionale, quando non ci siano espliciti richiami all’efficienza. Scrivere la soluzione di eventuali punti ‘teorici’ (esempio, esercizio **3.4**) semplicemente come commento nel codice Haskell.

1. Interdefinibilità di Funzionali

1. Definire il funzionale *zipWith* $f\ xs\ ys$ senza decomporre liste, ma usando un’espressione che contenga *zapp*, *f* ed eventualmente *xs* e *ys*.
2. Abbiamo visto che *zipWith* è più generale di *zip*. Tuttavia si può definire *zipWith* $f\ xs\ ys$ usando *zip* e un paio di altri funzionali visti a lezione.
3. Definire il funzionale *map* $f\ xs$ senza decomporre *xs*, ma usando un’espressione che contenga *foldr*, *f* e *xs*. Fare lo stesso usando *foldl*.
4. Argomentare brevemente sul perché non sia possibile definire *foldl* e *foldr* usando *map*.

2. Rimozione di Duplicati

Questo esercizio è parzialmente svolto nei problemi del “Voting System” e “Common Words” (vedi Lezioni **5** e **6**).

Nel `Prelude` di Haskell sono predefiniti i funzionali:

$$\begin{aligned} takeWhile &:: (a \rightarrow \text{Bool}) \rightarrow [a] \rightarrow [a] \\ dropWhile &:: (a \rightarrow \text{Bool}) \rightarrow [a] \rightarrow [a] \end{aligned}$$

dove *takeWhile* $p\ xs$ ritorna il segmento *iniziale* di valori di *xs* che soddisfano *p*, mentre *dropWhile* $p\ xs$ rimuove tale segmento iniziale. Esattamente come *take* e *drop* si ha che è sempre soddisfatta l’equazione algebrica: $xs = takeWhile\ p\ xs ++ dropWhile\ p\ xs$. Ad esempio:

```
Prelude> takeWhile (<3) [1,2,5,7,2,1]
[1,2]
Prelude> dropWhile (<3) [1,2,5,7,2,1]
[5,7,2,1]
```

1. Dare la definizione di *myTakeWhile* e *myDropWhile*;
2. scrivere una funzione ricorsiva *myRemoveDupsOrd* che rimuove i duplicati da una lista *ordinata* *xs* di lunghezza *n* in tempo $\mathcal{O}(n)$.
3. scrivere una funzione *myRemoveDups* che rimuove i duplicati da una *qualsiasi* lista *xs* di lunghezza *n* in tempo $\mathcal{O}(n \log n)$, preservando l'ordine originale delle prime occorrenze dei valori. Ad esempio:

```
Prelude> myRemoveDups [5,2,1,2,5,7,2,1,2,7]
[5,2,1,7]
```

3. Segmenti e sottoliste

1. Scrivere una funzione *prefissi* :: $[a] \rightarrow [[a]]$ che ritorna tutti i segmenti iniziali di una lista (vedi funzione *suffissi* nella Lezione 2).
2. Senza preoccuparsi dell'efficienza, ma usando i funzionali *prefissi*, *suffissi* e altri funzionali dello standard **Prelude**, scrivere una funzione *segSommaS* :: $(\text{Num } a) \Rightarrow [a] \rightarrow a \rightarrow [[a]]$ che data una lista numerica *xs* e un valore *s* restituisce tutti i segmenti (cioè sottoliste di elementi *consecutivi*) di *xs* di somma *s*.
3. Scrivere una funzione *sublSommaS* :: $(\text{Num } a) \Rightarrow [a] \rightarrow a \rightarrow [[a]]$ che data una lista numerica e un valore *s* restituisce tutte le sottoliste (anche di *elementi non consecutivi*) di somma *s*.

4. Ricorsione Primitiva in λ -calcolo (Facoltativo)

Nella lezione 7 abbiamo visto che i numerali di Church sono essenzialmente iteratori oltre che numeri. Sono, quindi, il funzionale *for* $f \ v \ n$ visto a lezione che applica la funzione *f* *n* volte a un valore iniziale *v* (calcola cioè quello che in matematica si scrive $f^n(v)$).

Tuttavia, sempre nella lezione 7 si parla del principio di *ricorsione primitiva* definito sotto, che è leggermente più generale:

$$\begin{aligned} \text{recP} &:: (\text{Integer} \rightarrow a \rightarrow a) \rightarrow a \rightarrow \text{Integer} \rightarrow a \\ \text{recP } f \ v \ 0 &= v \\ \text{recP } f \ v \ n &= f \ n \ (\text{recP } f \ v \ n - 1) \end{aligned}$$

Scrivere *recP* usando *for* e tuple (senza fare ricorsione sui naturali come sopra). Il che equivale a far vedere che posso definire la ricorsione primitiva in λ -calcolo usando numerali di Church e tuple.

5. Partizioni (Facoltativo)

Dato un numero intero positivo n , le *partizioni* di n sono tutti i modi in cui è possibile scrivere n come somma di altri numeri interi positivi.

Ad esempio, le partizioni di 4 sono le sequenze $[1,1,1,1]$, $[1,1,2]$, $[1,3]$, $[2,2]$, $[4]$. Sono considerate uguali partizioni che differiscono solo per l'ordine, quindi non vanno considerate nelle partizioni di 4 anche $[3,1]$ oppure $[1,2,1]$.

1. Scrivere una funzione Haskell `part :: Int → Integer`, che calcola il numero di partizioni di un certo numero n . Ad esempio, `part 4` calcola 5.

2. Se invece considero diverse tra loro anche partizioni che differiscono solo per l'ordine, quante sono?

3. Scrivere poi una funzione Haskell `parts :: Int → [[Int]]`, che calcola la lista delle partizioni di n . Ad esempio, `parts 4` calcola la lista $[[1,1,1,1], [1,1,2], [1,3], [2,2], [4]]$ (potrebbe ovviamente essere diverso l'ordine in cui si scrivono, ma suggerisco di seguire un ordine tanto nella generazione che nel conteggio).

4. (FACILE) Ma scrivere `part` usando `parts`? E la complessità è molto maggiore della `part` originaria?